Fuzz testing for input validation

The assigned Lab 7 starter (upstream commit 066cd16) parses ride durations, station codes and ride logs for the Bike Share Rental application used in Lab 6.

Method. Three Hypothesis tests generate valid and malformed inputs. Each checks the expected result for valid input and requires `None` for invalid input. The same tests ran against the starter and corrected source.

What failed? All three tests reported `AttributeError` when Hypothesis supplied `None`. The duration and station functions called `strip()` directly, and the log parser called `split()` directly. The original CLI also accepted 01:75, !12, and an unknown subscriber flag.

What changed? Each function checks for string input. The duration parser enforces minutes from 00 to 59. The station parser requires one ASCII letter followed by digits and preserves leading zeros. The log parser requires exactly three fields, a known bike type, a positive integer duration, and a yes/no flag. Invalid values return `None`, which the CLI displays as Invalid input.

What did fuzzing add? Hypothesis varied types, numeric boundaries and strings, then reported small counterexamples that made the original failures easy to reproduce. Each test also checks valid results so that simply rejecting every input fails the suite.

Result. The original run reported 3 failed in 1.69s. The corrected run reported 3 passed in 0.95s. The following pages contain the four Terminal captures and both complete Python source listings.

Application before and after

SS1 | Original application. The starter produces values for all three malformed entries.

```
(.venv) bvell@BhaveshVellurus-MacBook-Air L6_L7_53_bike_share_rental % cd ~/Documents/GitHub/L6_L7_53_bike_share_rental/tmp/lab7-original
(.venv) bvell@BhaveshVellurus-MacBook-Air lab7-original % printf '01:75\n!12\nelectric,45,maybe\n' | python bike_share_rental_parser.py
==== Running parse_ride_duration ====
Enter a ride duration as HH:MM (e.g. 01:15): Duration in minutes: 135

==== Running format_station_code ====
Enter a station code (e.g. a12): Formatted station code: !-12

==== Running parse_ride_log ====
Enter a ride log line as type,minutes,subscriber (e.g. electric,45,yes): Parsed ride log: ('electric', 45, False)
(.venv) bvell@BhaveshVellurus-MacBook-Air lab7-original %
```

SS3 | Corrected application. The same inputs each produce an Invalid input message.

```
(.venv) bvell@BhaveshVellurus-MacBook-Air lab7-original % cd ~/Documents/GitHub/L6_L7_53_bike_share_rental/LAB7
(.venv) bvell@BhaveshVellurus-MacBook-Air LAB7 % printf '01:75\n!12\nelectric,45,maybe\n' | python bike_share_rental_parser.py
Enter ride duration (HH:MM): Duration in minutes: Invalid input
Enter station code (e.g. a12): Formatted station code: Invalid input
Enter ride log (type,minutes,subscriber): Parsed ride log: Invalid input
(.venv) bvell@BhaveshVellurus-MacBook-Air LAB7 %
```

SS2 | Tests against the starter

```
(.venv) bvell@BhaveshVellurus-MacBook-Air lab7-original % python -m pytest -q --tb=short test_bike_share_rental_parser.py
FFF                                                                      [100%]
================================== FAILURES ===================================
_____________________________ test_parse_ride_duration _____________________________
test_bike_share_rental_parser.py:16: in test_parse_ride_duration
    @given(

test_bike_share_rental_parser.py:24: in test_parse_ride_duration
    assert parse_ride_duration(invalid) is None
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
bike_share_rental_parser.py:6: in parse_ride_duration
    hours, minutes = text.strip().split(':')
                     ^^^^^^^^^^
E   AttributeError: 'NoneType' object has no attribute 'strip'
E   Failing test case: test_parse_ride_duration(
E       # The test always failed when commented parts were varied together.
E       hours=0,  # or any other generated value
E       minutes=0,  # or any other generated value
E       invalid=None,  # or any other generated value
E       overflow=60,  # or any other generated value
E   )
_____________________________ test_format_station_code _____________________________
test_bike_share_rental_parser.py:29: in test_format_station_code
    @given(

test_bike_share_rental_parser.py:36: in test_format_station_code
    assert format_station_code(invalid) is None
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
bike_share_rental_parser.py:14: in format_station_code
    code = code.strip().upper()
           ^^^^^^^^^^
E   AttributeError: 'NoneType' object has no attribute 'strip'
E   Failing test case: test_format_station_code(
E       # The test always failed when commented parts were varied together.
E       zone='a',  # or any other generated value
E       number='0',  # or any other generated value
E       invalid=None,  # or any other generated value
E   )
_____________________________ test_parse_ride_log _____________________________
test_bike_share_rental_parser.py:41: in test_parse_ride_log
    @given(

test_bike_share_rental_parser.py:50: in test_parse_ride_log
    assert parse_ride_log(invalid) is None
           ^^^^^^^^^^^^^^^^^^^^^^^
bike_share_rental_parser.py:22: in parse_ride_log
    parts = line.split(',')
            ^^^^^^^^^^
E   AttributeError: 'NoneType' object has no attribute 'split'
E   Failing test case: test_parse_ride_log(
E       # The test always failed when commented parts were varied together.
E       bike='standard',  # or any other generated value
E       minutes=1,  # or any other generated value
E       subscriber=False,  # or any other generated value
E       invalid=None,  # or any other generated value
E   )
=========================== short test summary info ===========================
FAILED test_bike_share_rental_parser.py::test_parse_ride_duration - AttributeError: 'NoneType' object has no attribute 'strip'
FAILED test_bike_share_rental_parser.py::test_format_station_code - AttributeError: 'NoneType' object has no attribute 'strip'
FAILED test_bike_share_rental_parser.py::test_parse_ride_log - AttributeError: 'NoneType' object has no attribute 'split'
3 failed in 1.69s
(.venv) bvell@BhaveshVellurus-MacBook-Air lab7-original %
```

Three failing cases. Each parser raises AttributeError on None.

SS4 | Tests against corrected code

```
(.venv) bvell@BhaveshVellurus-MacBook-Air LAB7 % python -m pytest -q test_bike_share_rental_parser.py
...                                                                      [100%]
3 passed in 0.95s
(.venv) bvell@BhaveshVellurus-MacBook-Air LAB7 %
```

All three Hypothesis tests passed.

Source | Corrected parser

bike_share_rental_parser.py

```
"""Parse user-entered bike share details. Invalid input returns None."""

import re


def parse_ride_duration(text):
    """Convert hours and two-digit minutes to a total minute count."""
    if not isinstance(text, str):
        return None
    match = re.fullmatch(r"([0-9]{2,}):([0-5][0-9])", text.strip())
    if match is None:
        return None
    try:
        hours, minutes = map(int, match.groups())
    except ValueError:
        return None
    return hours * 60 + minutes


def format_station_code(code):
    """Format an ASCII zone letter and station number, preserving leading zeros."""
    if not isinstance(code, str):
        return None
    match = re.fullmatch(r"([A-Za-z])([0-9]+)", code.strip())
    if match is None:
        return None
    zone, number = match.groups()
    return f"{zone.upper()}-{number}"


def parse_ride_log(line):
    """Parse a known bike type, positive minute count and yes/no flag."""
    if not isinstance(line, str):
        return None
    fields = [field.strip() for field in line.split(",")]
    if len(fields) != 3:
        return None
    bike_type, minutes, subscriber = fields
    if bike_type not in {"standard", "electric", "cargo"}:
        return None
    if subscriber not in {"yes", "no"} or not re.fullmatch(r"[0-9]+", minutes):
        return None
    try:
        duration = int(minutes)
    except ValueError:
        return None
    if duration <= 0:
        return None
    return bike_type, duration, subscriber == "yes"


def main():
    """Run the three parser demonstrations."""
    prompts = (
        ("ride duration (HH:MM)", "Duration in minutes", parse_ride_duration),
        ("station code (e.g. a12)", "Formatted station code", format_station_code),
        ("ride log (type,minutes,subscriber)", "Parsed ride log", parse_ride_log),
    )
    for prompt, label, parser in prompts:
        try:
            value = input(f"Enter {prompt}: ")
        except EOFError:
            return
        result = parser(value)
        print(f"{label}: {result if result is not None else 'Invalid input'}")


if __name__ == "__main__":
    main()
```

Source | Hypothesis tests

test_bike_share_rental_parser.py

```python
"""Generate valid and malformed inputs for all three bike share parsers."""

from hypothesis import given, settings, strategies as st

from bike_share_rental_parser import (
    format_station_code,
    parse_ride_duration,
    parse_ride_log,
)

NON_TEXT = st.one_of(st.none(), st.integers(), st.booleans(), st.binary())
BIKE_TYPES = st.sampled_from(("standard", "electric", "cargo"))


@settings(derandomize=True, report_multiple_bugs=False)
@given(
    hours=st.integers(0, 999),
    minutes=st.integers(0, 59),
    invalid=st.one_of(NON_TEXT, st.text(alphabet="0123456789")),
    overflow=st.integers(60, 99),
)
def test_parse_ride_duration(hours, minutes, invalid, overflow):
    assert parse_ride_duration(f" {hours:02}:{minutes:02} ") == hours * 60 + minutes
    assert parse_ride_duration(invalid) is None
    assert parse_ride_duration(f"{hours:02}:{overflow}") is None


@settings(derandomize=True, report_multiple_bugs=False)
@given(
    zone=st.sampled_from(tuple("abcdefghijklmnopqrstuvwxyz")),
    number=st.text(alphabet="0123456789", min_size=1, max_size=12),
    invalid=st.one_of(NON_TEXT, st.text().map(lambda text: "!" + text)),
)
def test_format_station_code(zone, number, invalid):
    assert format_station_code(f" {zone}{number} ") == f"{zone.upper()}-{number}"
    assert format_station_code(invalid) is None
    assert format_station_code("") is None


@settings(derandomize=True, report_multiple_bugs=False)
@given(
    bike=BIKE_TYPES,
    minutes=st.integers(1, 10000),
    subscriber=st.booleans(),
    invalid=st.one_of(NON_TEXT, st.text(alphabet="0123456789")),
)
def test_parse_ride_log(bike, minutes, subscriber, invalid):
    flag = "yes" if subscriber else "no"
    assert parse_ride_log(f" {bike}, {minutes}, {flag} ") == (bike, minutes, subscriber)
    assert parse_ride_log(invalid) is None
    assert parse_ride_log(f"{bike},{minutes},maybe") is None
    assert parse_ride_log(f"{bike},{minutes},{flag},extra") is None
    assert parse_ride_log(f"{bike},0,{flag}") is None
    assert parse_ride_log(f"tandem,{minutes},{flag}") is None
```